**LAB 7: FreeRTOS**

## Part 1: Use the FreeRTOS

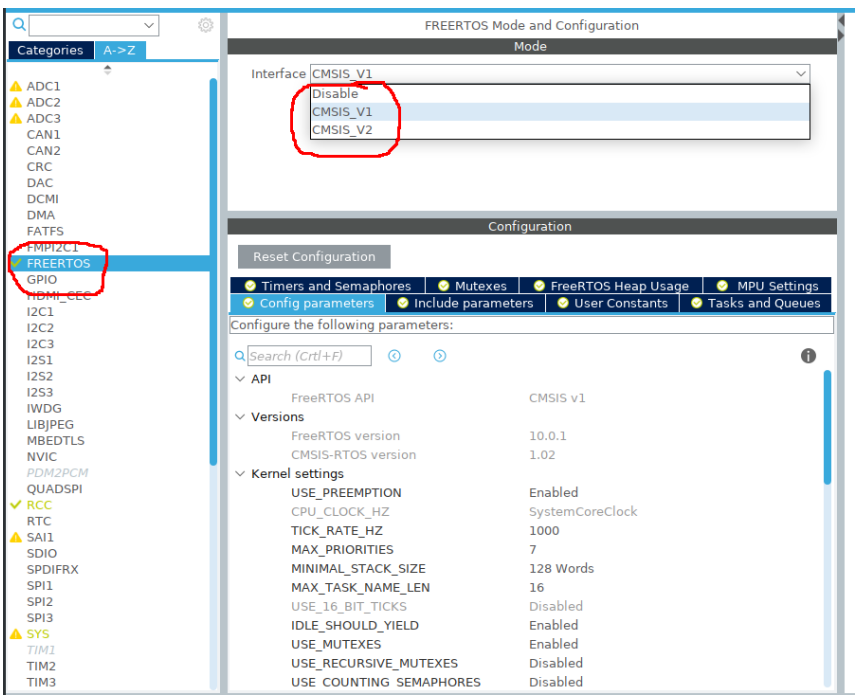
### 1.1. Objectives

- Learn how to use the FreeRTOS.
- Lear how to add tasks.
- Learn how to use queues
- Learn how to use semaphores.

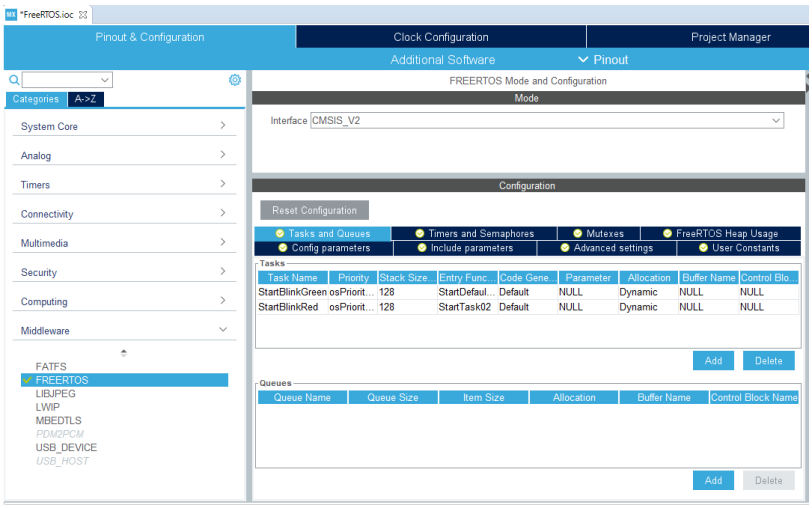
### 1.2. Create the project in STM32CubeIDE

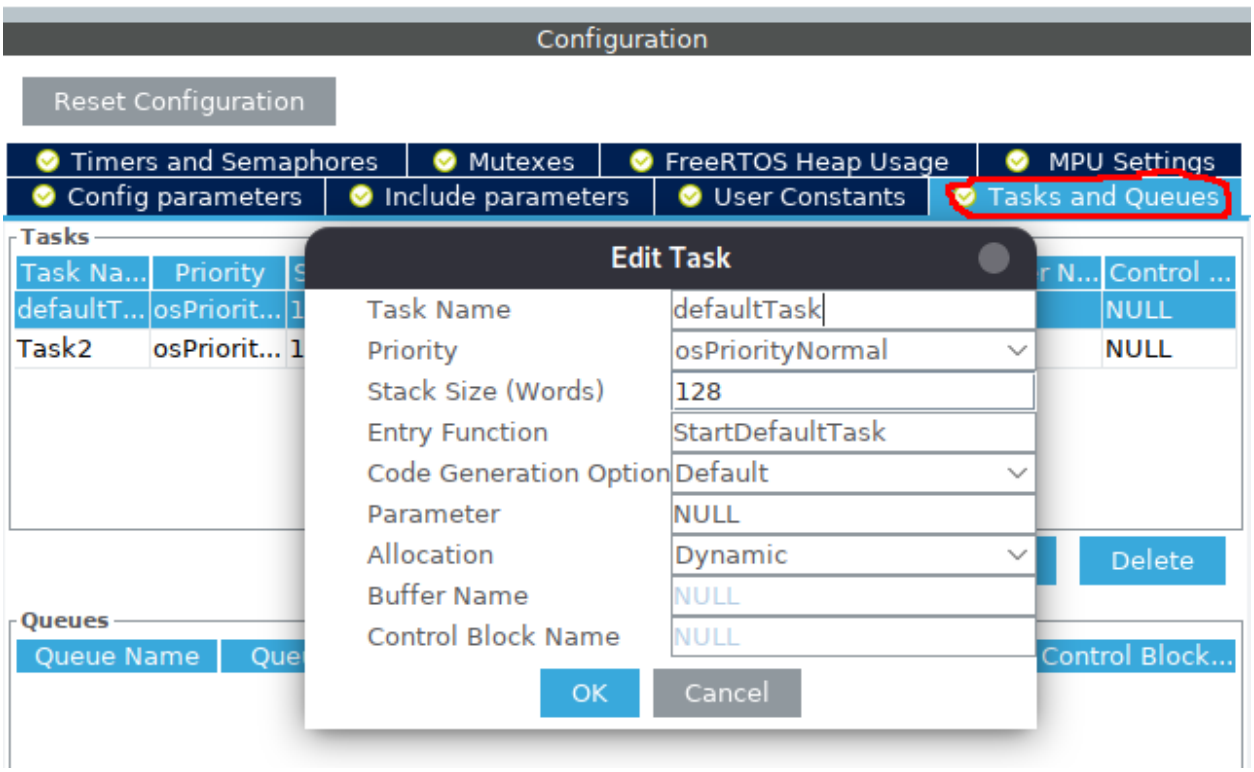
**Use same project used for the LCD and the MPU6050 (GY-521) in Lab 5**

# STM32CubeMX FreeRTOS Configuration




I am choosing version 2, because it is supported by many of STM32 devices.



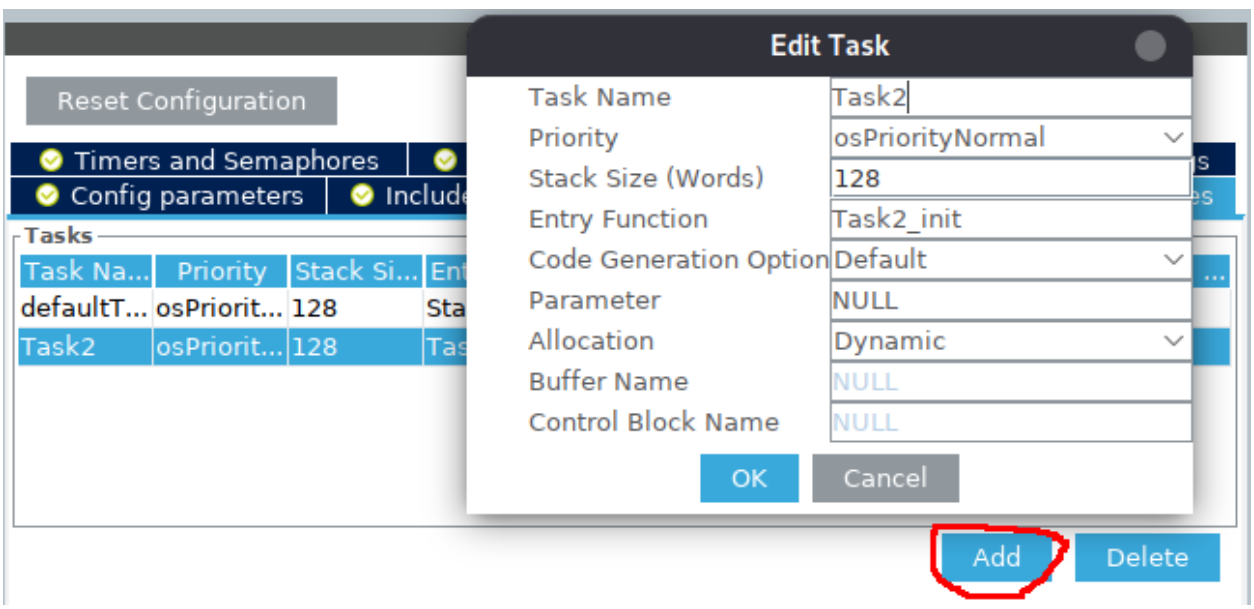




The Task details are explained below:

- **Task Name:** `defaultTask` – This is just the name of the task. You use this name in your code to work with this task.
- **Priority:** `osPriorityNormal` – This decides how important the task is. Normal priority means it runs after high-priority tasks but before low-priority ones. It helps the CPU know which task to run first.
- **Stack Size (Words):** `128` – This is the memory the task can use for its variables and function calls. Think of it as the workspace for the task.

- **Entry Function:** `StartDefaultTask` – This is the function that will run when the task starts. You put the task's code inside this function.
- **Code Generation Option:** `Default` – CubeMX will generate the task using its standard method.
- **Parameter:** `NULL` – If you wanted to give some information to the task when it starts, you could put it here. NULL means no information is passed.
- **Allocation:** `Dynamic` – Memory for the task is given at runtime. The other option is static, where memory is reserved before the program runs.

We will create another task for the test. Below is the image showing the task properties.



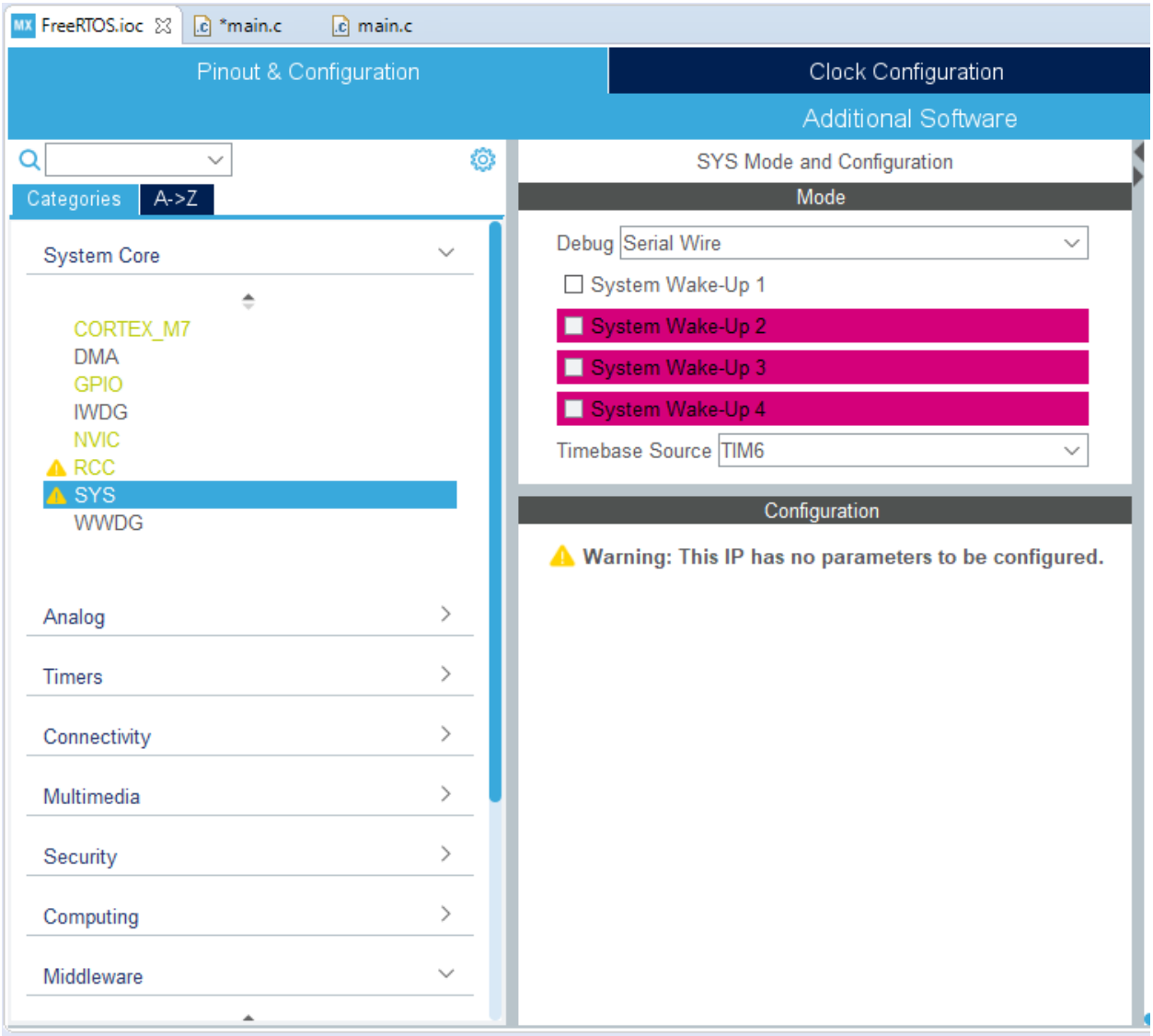

I am calling it **Task2**, with **normal priority**, **128** Stack Size and the entry function is defined as **Task2_init**. We will write this entry function in our code.

## System Timer

This project doesn't make use of any timers yet but taken into consideration for the future purpose of this project a basic timer will be used. Based on the datasheet of the

Nucleo, timer 6(TIM6) and timer 7(TIM7) are the basic 16-bit timers. To prevent conflicts in the future the timebase source will be TIM6.

To do this open the category “System Core” and open “SYS”. Here we can change the timebase source to TIM6.



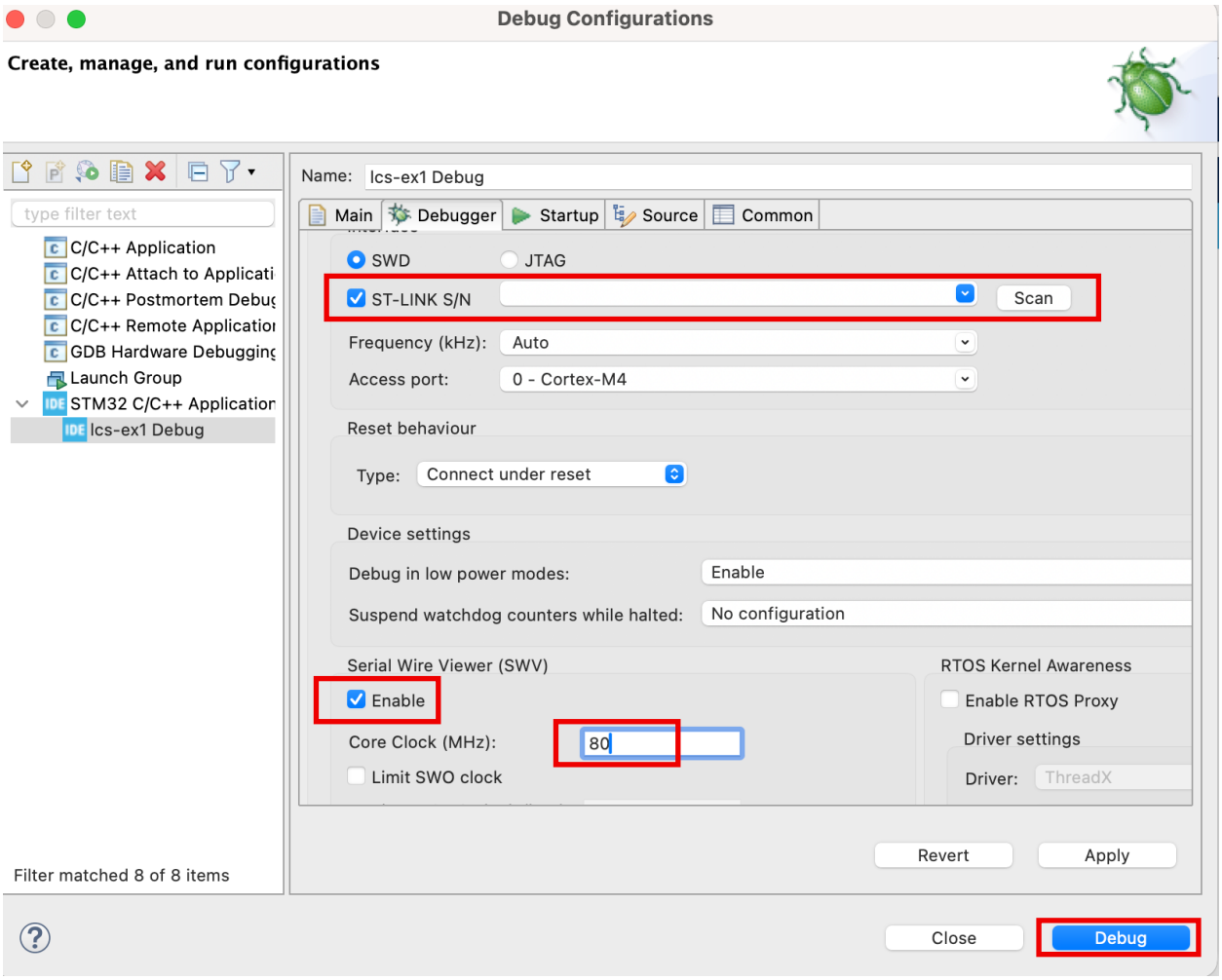

## Write the default function

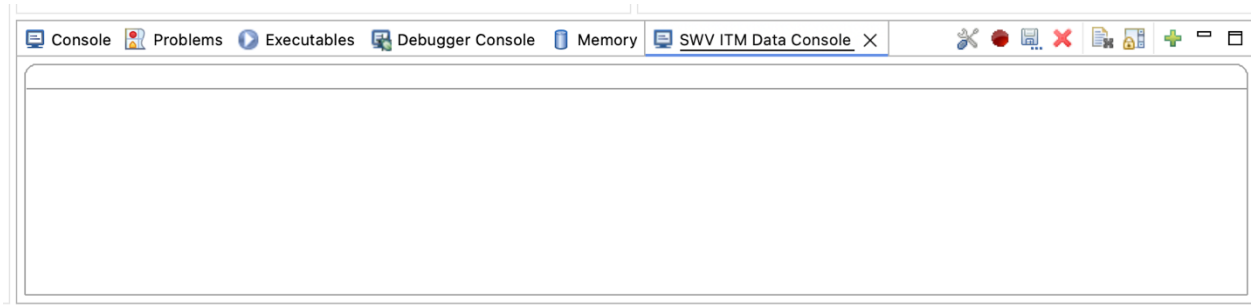
To toggle the LEDs, the software will make use of HAL_GPIO_TogglePin() command. To show that the system can run multiple tasks at the same time we will use a delay to toggle the LEDs. The command osDelay() will be used to create a delay. Without an RTOS the delays should conflict the frequency of the LEDs toggling on and off. For this test the green LED will toggle every second and the red LED will toggle every half

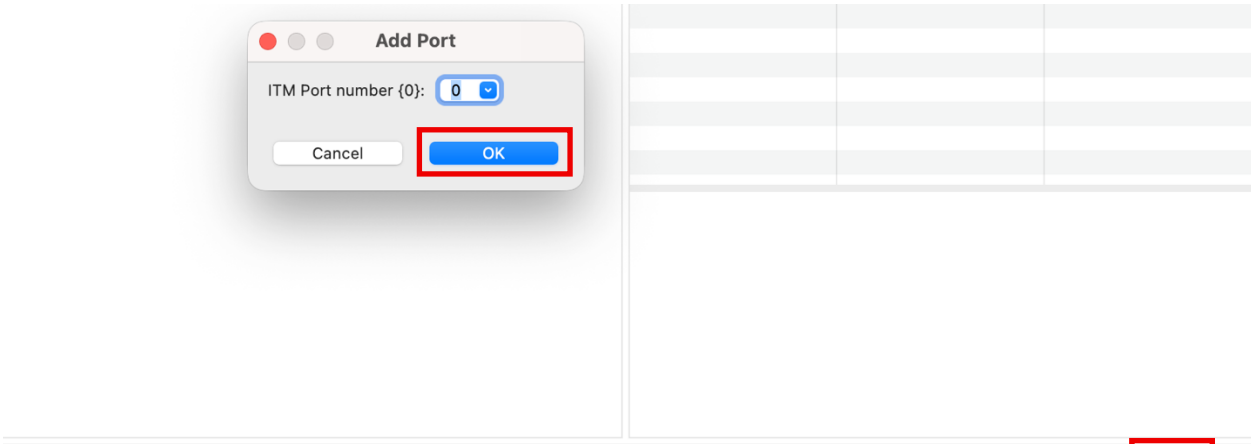
second. The image below shows the implementation for toggling the pins and the used delays.

```
/* USER CODE END Header_StartDefaultTask */
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
    HAL_GPIO_TogglePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin);
    osDelay(500);
  }
  /* USER CODE END 5 */
}
```

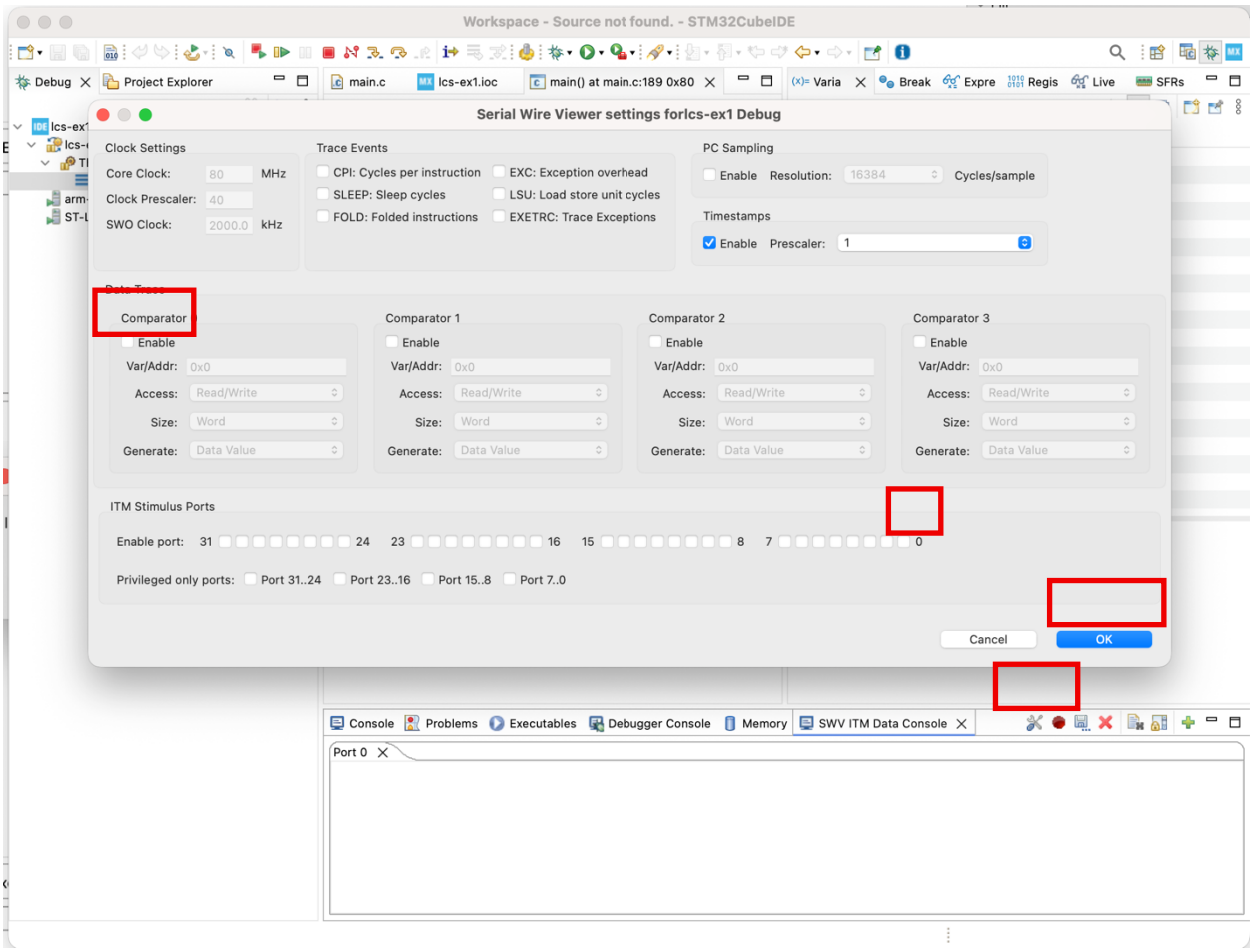
To enable the Serial Debugger

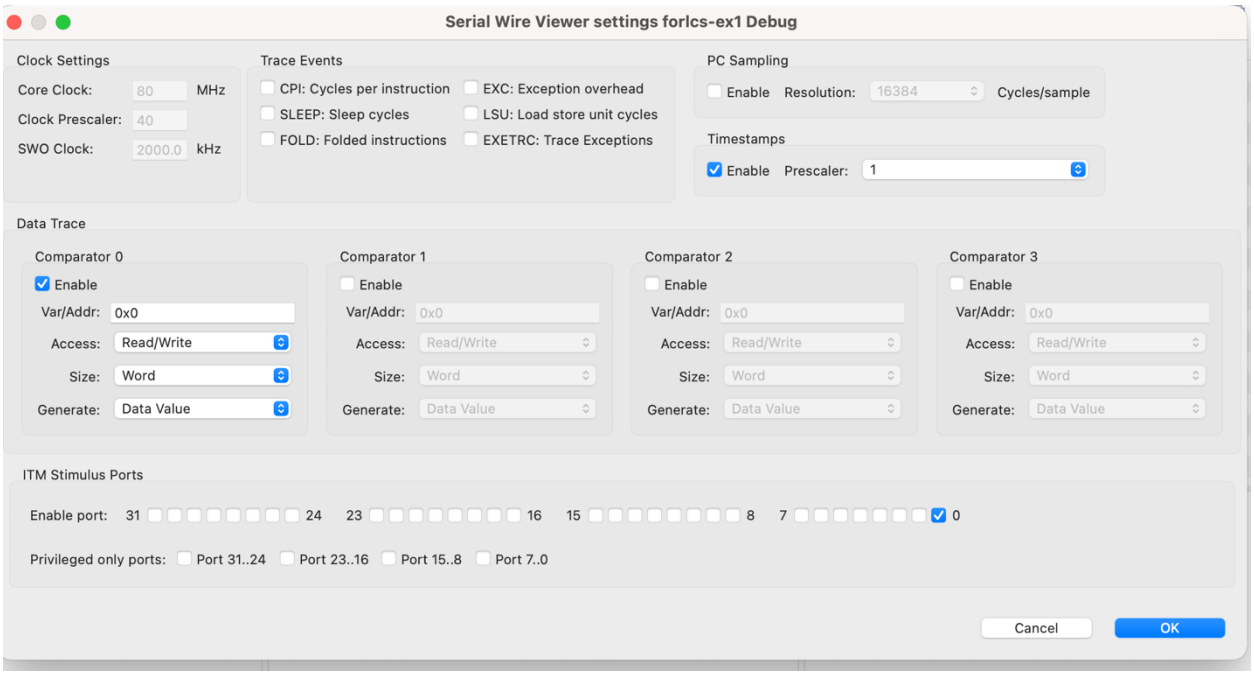

Debug Configurations
Create, manage, and run configurations
type filter text
C/C++ Application
C/C++ Attach to Applicati
C/C++ Postmortem Debug
C/C++ Remote Applicatio
GDB Hardware Debugging
Launch Group
STM32 C/C++ Applicatio
lcs-ex1 Debug
Filter matched 8 of 8 items
Name: lcs-ex1 Debug
Main
Debugger
Startup
Source
Common
SWD
JTAG
ST-LINK S/N
Scan
Frequency (kHz): Auto
Access port: 0 - Cortex-M4
Reset behaviour
Type: Connect under reset
Device settings
Debug in low power modes: Enable
Suspend watchdog counters while halted: No configuration
Serial Wire Viewer (SWV)
Enable
Core Clock (MHz): 80
Limit SWO clock
RTOS Kernel Awareness
Enable RTOS Proxy
Driver settings
Driver: ThreadX
Revert
Apply
Close
Debug


Console
Problems
Executables
Debugger Console
Memory
SWV ITM Data Console


Add Port
ITM Port number {0}:
0
Cancel
OK

Console
Problems
Executables
Debugger Console
Memory
SWV ITM Data Console


Workspace - Source not found. - STM32CubeIDE
Serial Wire Viewer settings forlcs-ex1 Debug
Clock Settings
Core Clock: 80 MHz
Clock Prescaler: 40
SWO Clock: 2000.0 kHz
Trace Events
CPI: Cycles per instruction
EXC: Exception overhead
SLEEP: Sleep cycles
LSU: Load store unit cycles
FOLD: Folded instructions
EXETRC: Trace Exceptions
PC Sampling
Enable Resolution: 16384 Cycles/sample
Timestamps
Enable Prescaler: 1
Comparator 0
Comparator 1
Comparator 2
Comparator 3
Enable
Var/Addr: 0x0
Access: Read/Write
Size: Word
Generate: Data Value
ITM Stimulus Ports
Enable port: 31 24 23 16 15 8 7 0
Privileged only ports: Port 31..24 Port 23..16 Port 15..8 Port 7..0
Cancel
OK
Console
Problems
Executables
Debugger Console
Memory
SWV ITM Data Console
Port 0




Task Action code example

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  ITM_SendChar(message);
  ITM_SendChar('\n');
}
```

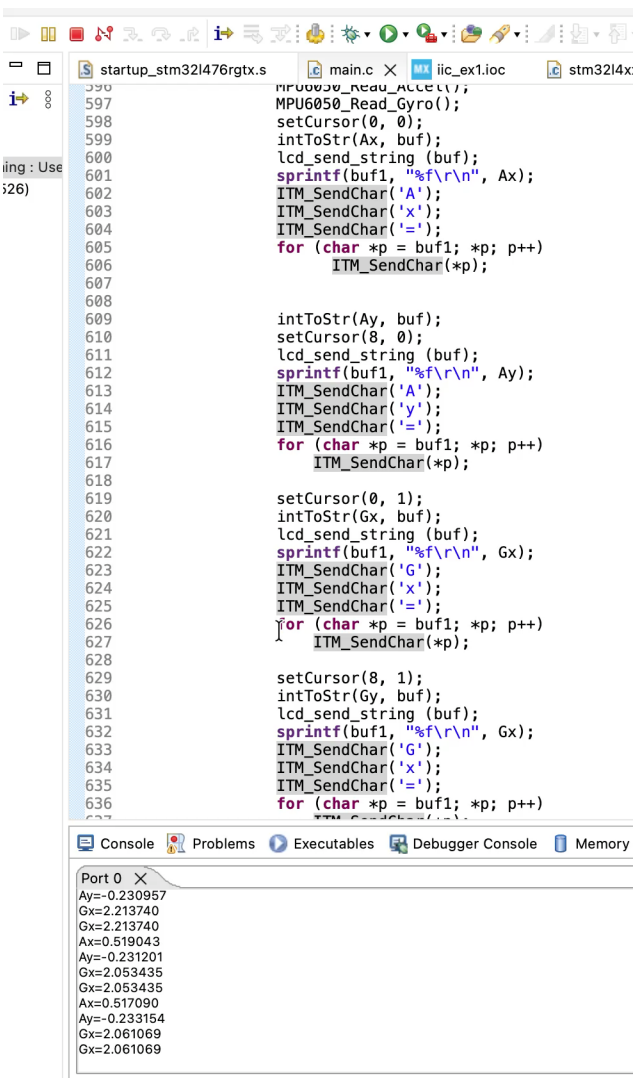
For task 2 add the following code

```
void StartTask02(void *argument)
{
  /* USER CODE BEGIN StartTask02 */
     char buf[10];
     char buf1[16];


                /* USER CODE END PV */


                lcd_init();
```

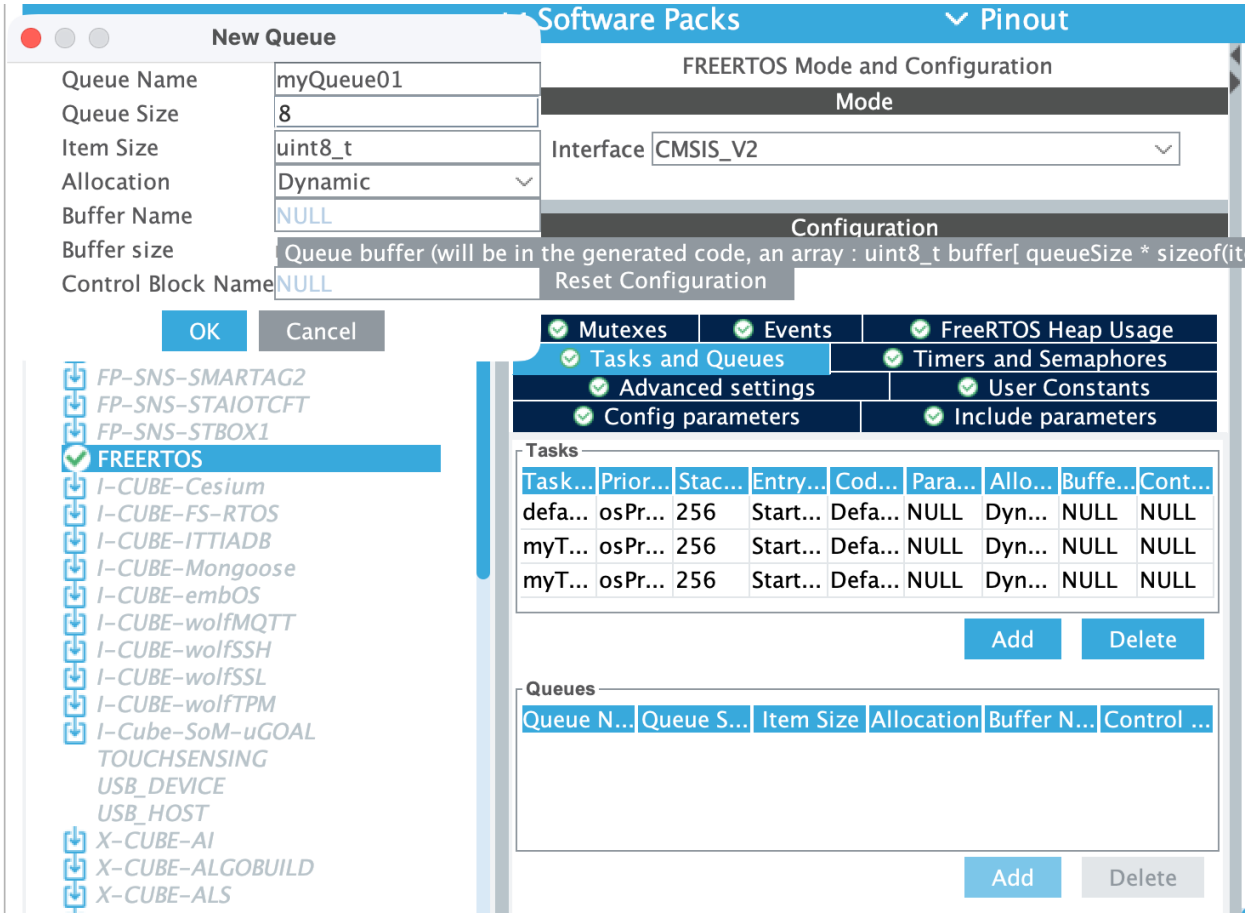
```c
/* Infinite loop */
for(;;)
{


                    MPU6050_Read_Accel();
                   MPU6050_Read_Gyro();
                    setCursor(0, 0);
                    intToStr(Ax, buf);
                    lcd_send_string (buf);
                    sprintf(buf1, "%f\r\n", Ax);
                    ITM_SendChar('A');
                    ITM_SendChar('x');
                   ITM_SendChar('=');
                    for (char *p = buf1; *p; p++)
                         ITM_SendChar(*p);


                    intToStr(Ay, buf);
                    setCursor(8, 0);
                    lcd_send_string (buf);
                    sprintf(buf1, "%f\r\n", Ay);
                    ITM_SendChar('A');
                    ITM_SendChar('y');
                    ITM_SendChar('=');
                    for (char *p = buf1; *p; p++)
                         ITM_SendChar(*p);

                    setCursor(0, 1);
                    intToStr(Gx, buf);
                    lcd_send_string (buf);
                    sprintf(buf1, "%f\r\n", Gx);
                    ITM_SendChar('G');
                    ITM_SendChar('x');
                    ITM_SendChar('=');
                    for (char *p = buf1; *p; p++)
                         ITM_SendChar(*p);

                    setCursor(8, 1);
                    intToStr(Gy, buf);
                    lcd_send_string (buf);
                    sprintf(buf1, "%f\r\n", Gx);
                    ITM_SendChar('G');
```

```
                    ITM_SendChar('x');
                    ITM_SendChar('=');
                    for (char *p = buf1; *p; p++)
                         ITM_SendChar(*p);

                    osDelay(500);
                   lcd_clear();

  }
  /* USER CODE END StartTask02 */
}
```

Output should be like this



**Part 2: Add the Queue**




**The following lines should appear in your code**

```
/* Definitions for myQueue01 */
osMessageQueueId_t myQueue01Handle;
const osMessageQueueAttr_t myQueue01_attributes = {
  .name = "myQueue01"
};
/* creation of myQueue01 */
  myQueue01Handle = osMessageQueueNew (8, sizeof(uint8_t),
&myQueue01_attributes);
```

**Now replace the code in the tasks by the following**

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */
     uint8_t x=0;
```

```
  /* Infinite loop */
  for(;;)
  {
    HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_5);
    Task_action('S');
    osMessageQueuePut(myQueue01Handle,&x,0,200);
    if(++x>9) x=0;
       osDelay(1000);

  }
  /* USER CODE END 5 */
}
void StartTask02(void *argument)
{
  /* USER CODE BEGIN StartTask02 */

uint8_t res=0;

/* USER CODE END PV */

/* Infinite loop */
for(;;)
{

Task_action('R');
osMessageQueueGet(myQueue01Handle, &res, NULL, 2000);
Task_action(res+48);

  }
  /* USER CODE END StartTask02 */
}
```

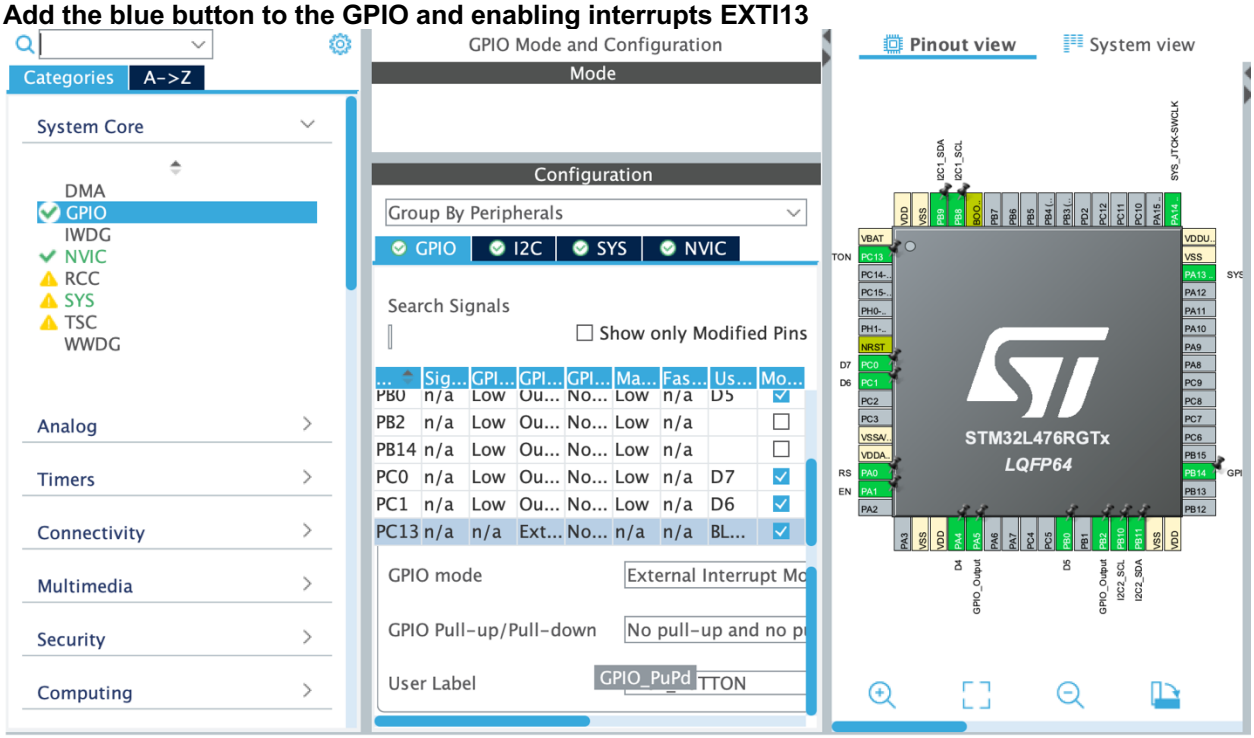
**The output should be something like**

Port 0

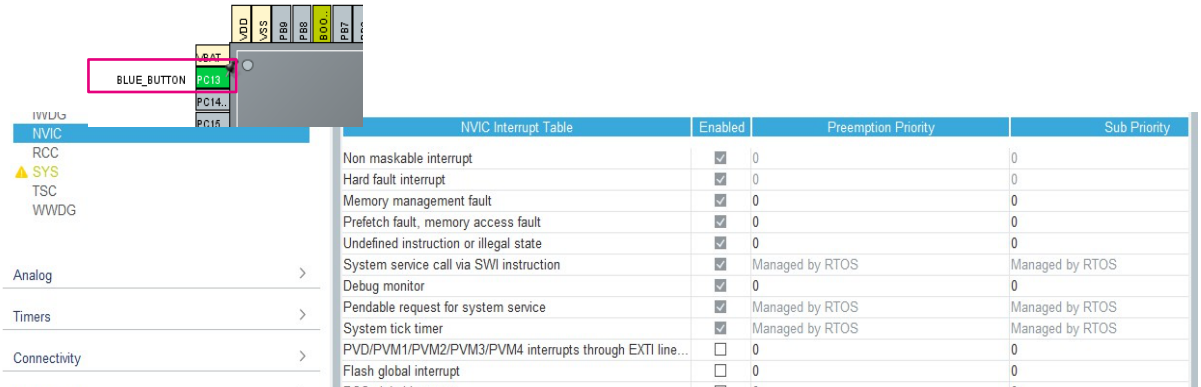
R
S
7
R
S
8
R
S
9
R

## Part 3: Use the Semaphores

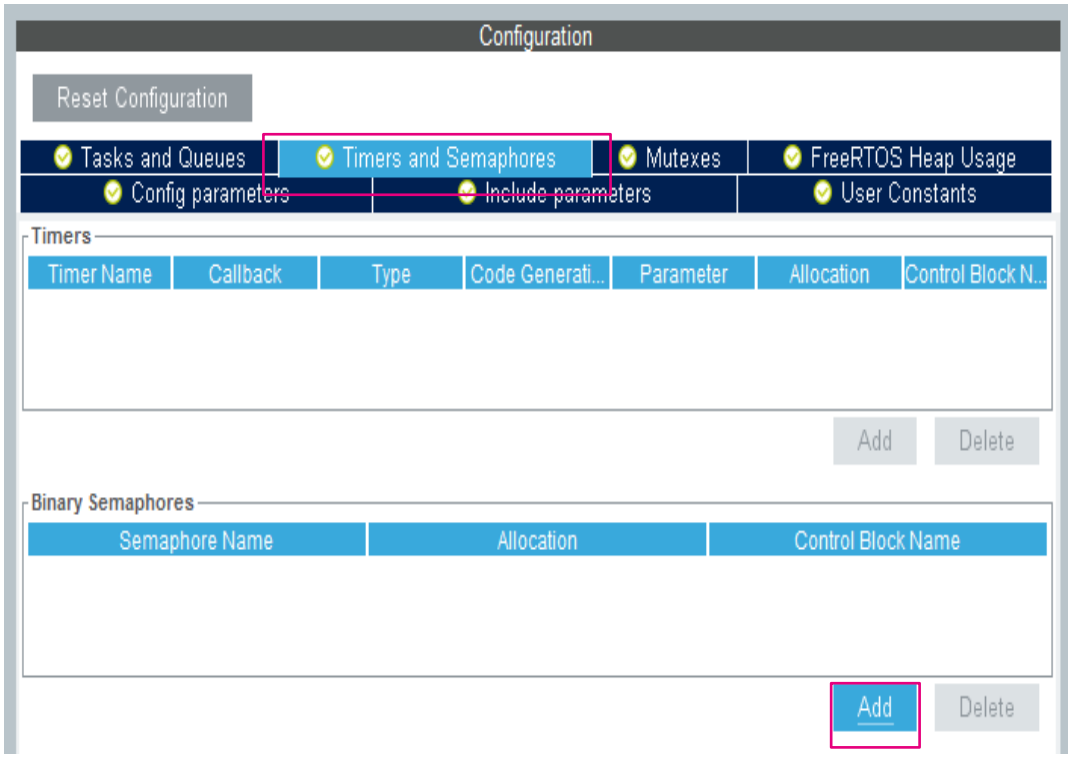
### Add the blue button to the GPIO and enabling interrupts EXTI13

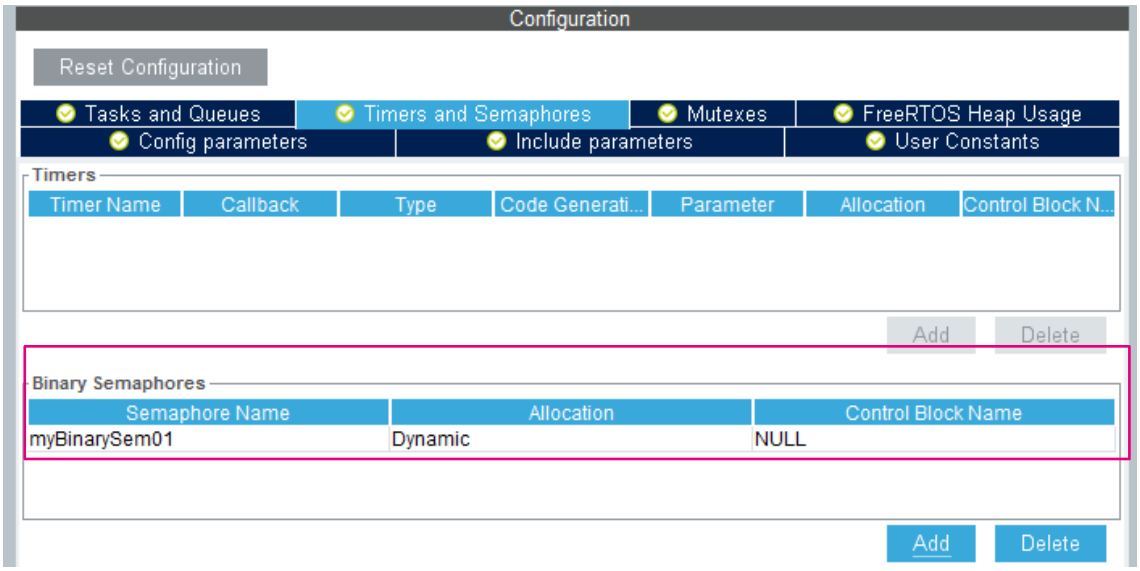


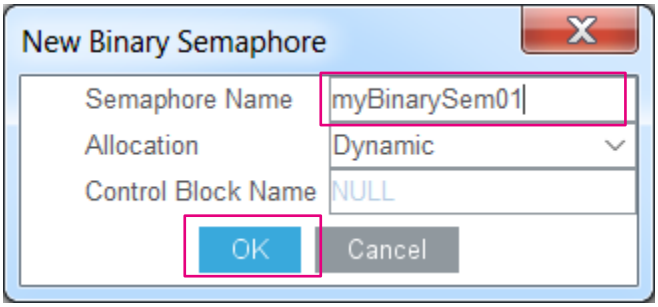

### Enable interrupts



- Create binary semaphore
- Select Timers and Semaphore tab
- Click Add button in Binary Semaphores section
- Set name: myBinarySem01
- Click OK button






You should see the following code generated

```
/* Definitions for myBinarySem01 */
osSemaphoreId_t myBinarySem01Handle;
const osSemaphoreAttr_t myBinarySem01_attributes = {
  .name = "myBinarySem01"
};


/* Create the semaphores(s) */
  /* creation of myBinarySem01 */
  myBinarySem01Handle = osSemaphoreNew(1, 1,
&myBinarySem01_attributes);
```

Change your code of tasks to be

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */


  /* Infinite loop */
  for(;;)
  {
       osDelay(2000);
       Task_action('1'); //Task1 release semaphore
       osSemaphoreRelease(myBinarySem01Handle);


  }
  /* USER CODE END 5 */
```

```
}

void StartTask02(void *argument)
{
  /* USER CODE BEGIN StartTask02 */

  /* Infinite loop */
  for(;;)
  {

       osSemaphoreAcquire(myBinarySem01Handle,4000);
       Task_action('2'); //Task2 synchronized
  }
  /* USER CODE END StartTask02 */
}
```

Add the callback function for interrupts

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
Task_action('!');
osSemaphoreRelease(myBinarySem01Handle);
}
```

The output should be something like

**Part 4: Use the Queues and semaphores with the LCD and 2 sensors of your choice**

**This should be reading the values from the sensors and displaying outputs on the LCD.**

**First sensor displays on the first line of the LCD while the second sensor displays on the second line of the LCD.**

**Submit the whole project + a video showing the operation**

Function for integer to string if needed

```
void intToStr(int num, char *str) {
    int i = 0;
    int isNeg = 0;

    if (num == 0) {
        str[i++] = '0';
        str[i] = '\0';
        return;
    }

    if (num < 0) {
        isNeg = 1;
        num = -num;
    }

    while (num != 0) {
        str[i++] = (num % 10) + '0';
        num /= 10;
    }

    if (isNeg)
        str[i++] = '-';

    str[i] = '\0';

    // Reverse the string
    for (int j = 0; j < i / 2; j++) {
        char t = str[j];
        str[j] = str[i - j - 1];
        str[i - j - 1] = t;
    }
```

}